

Mainquns

Конвейер автономного прогрева антидетект-сессий

Linken Sphere 2 × Windows VPS × AI-driven development

Handover Document · Июнь 2026

Версия 1.0

01 · Проблема

Прогрев browser-сессий (cookies, history, fingerprint maturity) — фундамент работы любой антидетект-инфраструктуры. Ручной прогрев одной сессии требует 4-9 часов кликов через RDP: оператор открывает Linken Sphere, логинится, импортирует сессию, активирует API, гоняет 300-500 URL через warmup-фичу. Это не масштабируется: десять VPS = десять параллельных RDP-сессий, постоянное вмешательство, риск ошибки оператора, простои.

Стандартные подходы ломаются на:

Проблема	Что ломает
Windows Update reboot	Прерывает прогрев, потеря состояния
Kernel-Power 41 crash	VPS падает в фоне, RDP не отвечает
LS Electron смерть mid-cycle	Connection refused, нужен manual restart
8+ VPS на одном LS-аккаунте	409 conflict при одновременном доступе к сессии
FHD-render на дешёвых VPS	Template matching падает с 0.99 до 0.4
LS cloud sync 5-7 минут	Mass Import висит, оператор думает что зависло

02 · Решение

Mainquns — полностью автономный пайплайн прогрева. Одна команда PowerShell на новой VPS → 10-15 минут setup → 3-9 часов автономной работы без оператора → файл с 5-7k cookies + звуковой push на телефон ☎.

Ключевые свойства:

☐ Zero RDP-вмешательство

После старта оператор не входит в VPS пока не прилетит финальный push.

☐ Self-healing на 6 слоёв

Boot recovery, watchdog, API ping, 409 pyramid, FHD fallback, Task Scheduler retry.

☐ Push-driven observability

ntfy.sh: ⚙️/🔔/📱/📧 успехи, ⚠️ алерты с встроенной командой recovery.

☐ Парк-friendly

Anti-collision jitter 0-120с, 9-step retry pyramid под 8+ VPS на одном LS-аккаунте.

☐ Survival mode

Переживает Kernel-Power 41, Windows Update, hard reset, network flap, hostside cache.

☐ Fingerprint variability

8 видеокарт × профили CPU/RAM/Screen × Windows 10/11 — реалистичные конфиги без абсурдов (RTX 4070 + 4 ядра).

03 · Архитектура — 6 слоёв защиты

Пайплайн построен как luxury sake — каждый слой решает свой класс сбоев. Если падает один — следующий подхватывает в течение минут.

	Слой	Что делает
Layer 6	Stuck detection	Watchdog мониторит LastRunTime главной задачи. Если >75 мин не тикала
Layer 5	API ping	5-мин watchdog ping LS API. 3 fail подряд (~15 мин) → kill LS + ls_launch.bat. [
Layer 4	Self-recovery	warmup_api ловит connection refused в цикле → kill+launch LS + retry. Max 2
Layer 4.5	Initial ping retry	5 попыток с backoff [5,10,15,20]с (~50с) на старте цикла. Переживает transi
Layer 3	9-step pyramid	Retry 409 conflict с backoff [10,30,60,120,240,360,force_stop,360,480]с. Покры
Layer 2	Task Scheduler	Time (45 мин) + AtStartup (PT2M delay). MultipleInstances=IgnoreNew защища
Layer 1	Boot recovery	AutoAdminLogon → Startup folder ls_launch.bat (cwd /D для evo:// protocol) → n
Layer 0	Preventive	NoAutoRebootWithLoggedOnUsers, lockouthreshold=0, firewall preauth BCEX L

04 · Жизненный цикл одной VPS

```
[OPERATOR]
| iex setup.ps1 + 3 env vars (LS_EMAIL, LS_PASSWORD, NTFY_TOPIC)
▼
[SETUP] 10-15 мин
| Git/Python/Repo/install.bat/Task Scheduler
▼
[UI INSTALL] warmup.py – 5-10 мин (single-shot)
| ensure_LS → login + Autologin → API port → session import
| → wait IMPORT (до 600с) → polling /sessions (до 460с)
| ☐ push «RDP можно отключать»
▼
[CYCLES] warmup_api.py – каждые 45 мин Task Scheduler
| ping retry (5×[5,10,15,20]с) → signin → 7 чанков × 14 URL
| → ☐ push после каждого успешного цикла
|
| ▼ (3-9 циклов до target = random(300, 500))
|
[FINISH] target reached
| schtasks /change /disable LinkenSphereWarmup
| LsWatchdog self-disable при .notified_done
| export cookies → ☐ push (priority=HIGH, звуковой)
▼
[IDLE] оператор забирает cookies, списывает VPS
```

05 · Метрики разработки

Проект разработан в AI-driven формате: оператор + Claude Code (Opus 4.7/4.8) + параллельные subagent'ы. Работа велась с 14 мая по 16 июня 2026 — 34 дня подряд без перерывов, по 8-12 часов в день. Ниже — данные из git log и оценки по локальным транскриптам.

50	34	~7 500
коммитов в main (публичная часть)	дня подряд без перерыва (14 мая → 16 июня 2026)	строк итога (5700 кода + 1800 документации)

~340	~300-500M	~1500-2500
часов оператор-AI сессий (34 дня × 8-12 ч)	токенов оплачено* + ~2-3B cache reads	запусков subagent'ов* (Explore / Plan / Code Review)

* Токены и subagent-запуски — оценка экстраполяцией по локальному транскрипту одной сессии (16 июня: ~5M оплаченных токенов и 17 subagent'ов за ~3.5 часа) на ~340 часов работы. Точные цифры — в Anthropic Console.

06 · Как использован AI

Mainquns — кейс parallel AI-pair programming с Claude Code. Оператор задавал направление и ловил edge case'ы на реальных VPS; Claude писал код, документацию, разруливал диагностику.

Распределение задач:

Задача	Кто делал
Архитектурный дизайн (6 слоёв защиты)	AI ← Operator
Написание Python / PowerShell / Bat кода (~5700 строк)	AI
Регенерация template-шаблонов под FHD (13 файлов)	Operator (скриншоты)
Документация (CLAUDE.md, CONFIG_GUIDE, PIPELINE, README)	AI
Эмпирический сбор edge cases (E-01..E-22 + F-01..F-12)	Operator (через RDP) → AI
Стресс-тест архитектуры (10 findings)	AI subagent (Explore)
Диагностика на live VPS (логи, push'и, ntfy)	Operator → AI
Эта презентация и handover-документация	AI

Ключевые AI-фичи в работе:

- Subagent parallelism — пока главный Claude писал код, фоновые Explore-агенты крутили аудит, поиск конфликтов в таймингах, перепроверяли архитектуру.
- Long-running CLAUDE.md — файл-контекст 48 КБ, который автоматически подгружается в каждую новую сессию. Без него каждый раз пришлось бы заново объяснять архитектуру.
- Push-driven feedback loop — операторские push-сообщения от ntfy → анализ → код → push.
- Empirical edge case logging — 34 промаркированных кейса (E-01..E-22 + F-01..F-12) в документации. AI сохраняет всё в текст, не теряет между сессиями.

07 · Цифры

Производительность одной VPS:

Параметр	Значение
Setup time (с нуля)	10-15 минут
UI install (первый запуск warmup.py)	5-10 минут (worst-case ~34 мин)
Один цикл прогрева (warmup_api)	~37 минут (worst-case ~50 мин)
Total time до target=500	4-9 часов
URL прогревается за цикл	95-100 (7 чанков × 14)
Cookies на финише	5 000 – 7 000
URL pool	40 144 уникальных
XLSX fingerprint variations	8 GPU × CPU/RAM/Screen × 2 Win = ~96 комбо

Парк-готовность:

Параметр	Поведение
8+ VPS на одном LS-аккаунте	9-step retry pyramid разруливает 409 conflict
Anti-collision jitter	0-120с randomized задержка при старте цикла
Resolution support	2560×1440 (primary) + 1920×1080 (FHD fallback)
FHD render quirk	13 _fhd.png templates + numbered variants
Survives reboot	AutoAdminLogon + Startup folder + Task AtStartup
Survives Windows Update	NoAutoRebootWithLoggedOnUsers=1
Survives Kernel-Power 41	Boot recovery + notify_boot.py +  push

08 · Что в этом архиве

Папка	Содержимое
01_repo/	Полный snapshot кода (70 файлов, ~3.4 MB). Готов к git init + push.
02_docs/CLAUDE.md	Главный архитектурный документ (48 KB, 14 разделов).
02_docs/CONFIG_GUIDE.md	Гайд по операторским правкам (URL target, fingerprint, timings).
02_docs/CONNECTION_GUIDE.md	Как новому владельцу подключиться к проекту.
02_docs/OPERATOR_GUIDE.md	Daily инструкция оператора (push'и, recovery, дисциплина).
02_docs/PIPELINE.md	Старая pipeline-документация (опционально).
02_docs/README.md	Repo intro.
02_docs/ИНСТРУКЦИЯ.md	User-facing инструкция (RU).
02_docs/architecture.pdf	Deep Dive с диаграммами (v3.0, 470 KB) — если есть.
03_dialogs/	Транскрипты Claude Code сессии (JSONL, ~5 MB).
04_presentation/	Этот PDF + источник Markdown.

Куда смотреть в первую очередь:

1. README_FIRST.md (корень архива) — общая ориентация
2. 02_docs/CONNECTION_GUIDE.md — как получить доступы
3. 02_docs/OPERATOR_GUIDE.md — daily workflow
4. 02_docs/CLAUDE.md — глубокая архитектура (час чтения)

09 · Куда развивать дальше

Опциональные улучшения, не блокирующие работу:

- Объединить дубль push'ей при LS recovery failed (F-10 в CLAUDE.md). Сейчас за один fail приходят два △.
- Различить polling-timeout от import-timeout в warmup.py — разные recovery-команды для разных причин.
- Виртуальный display driver для FHD-locked VPS (некоторые дешёвые провайдеры не отдадут >1080p).
- Метрики на dashboard — экспорт ntfy push'ей в Grafana / Prometheus.
- Авто-замена сдохших VPS — интеграция с API провайдеров (Hetzner, Vultr) для авто-redeploy.
- A/B тест fingerprint'ов — добавить tracking какие комбинации видеокарт/CPU дают самый «человеческий» результат на anti-fraud тестерах.

Что НЕ нужно трогать без понимания:

- △ PNG-template thresholds (0.80 для большинства, 0.76 для skip).
- △ Тайминги Layer 1-6 — все consciously conservative, эмпирически проверены.
- △ 9-step retry pyramid — покрывает 8+ VPS, не сжимать.
- △ urls_per_chunk_max=14 — выше → LS API 422.
- △ AutoAdminLogon + AtStartup PT2M — критично для recovery после reboot.

Спасибо.

Mainquns — пример того, что получается когда AI-pair programming встречается с реальной production-задачей: десятки edge cases, флакающие VPS, медленные cloud sync'и и неотлаженный LS API. За 34 дня — рабочий конвейер, который не требует оператора после старта.

Версия 1.0 · Июнь 2026 · r2d23c/mainquns